

0.90.4. L'écran titre — tout l'écran glisse

On repart du 0.90.1 : cette fois, c'est TOUT l'écran qui glisse vers la droite. `defiler(d, 0)`, et `d` qui diminue à chaque image.

1. Ce qu'il faut comprendre

On repart du 0.90.1 (sans le B du 0.90.2), avec une seule chose en plus : **tout l'écran** glisse de gauche à droite, « C EST PARTI » compris.

Ce qui est nouveau ici : `defiler(x, y)`. Il ne bouge pas une lettre : il déplace **la caméra** qui regarde le décor.

`x` dit de combien de **pixels** la caméra est poussée vers la droite.

Pour que le décor aille à DROITE, la caméra va à GAUCHE : comme dans un train, quand tu avances, le paysage recule. Alors `d` **diminue** : `d--`, un pixel à chaque image.

`d` **passé sous 0** ? C'est un `uint8_t` : après 0 vient **255**, puis 254... (l'octet qui boucle, la leçon 35). Et le décor fait justement **256 pixels** de large : il revient tout seul, sans fin. Ce qui sort par la droite réapparaît à gauche.

60 images par seconde, un pixel chaque fois : le texte traverse l'écran (160 pixels) en moins de 3 secondes, tout en douceur.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1465 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	62 o	← la console, à l'allumage — la mise en route de la console
texte_grand_0()	195 o	← main() — ta fonction
main()	120 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran

```

LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           848 o ← CopierTuiles – la police entière, et 9 tuiles dessinées
"APPUIE SUR START" 16 o ← texte() – un texte gravé
"C EST PARTI"      11 o ← texte() – un texte gravé
GRAND_TABLE_0_0_0  12 o ← poser() – une table gravée
GRAND_TABLE_0_1_1  12 o ← poser() – une table gravée
GRAND_TABLE_0_2_2  12 o ← poser() – une table gravée
GRAND_TABLE_0_3_3  12 o ← poser() – une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    poser()      × 4    122 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o),
GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
    texte()      × 2    22 o sur place, dans main() + EcrireTexte (10 o)
    effacer()    × 1    14 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
    couleurFond() × 1    12 o sur place, dans main()
    defiler()    × 1    7 o sur place, dans main()
    bouton()     × 1    5 o sur place, dans main() + LireManette (37 o)
    image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
    texte_grand_0() × 1    3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <couleurTexte> // choisit la couleur des lettres
#include <texteGrand>   // écrit un texte agrandi, taille de 0 à 10
#include <texte>         // écrit un texte à l'écran
#include <bouton>        // lit un bouton de la manette
#include <effacer>       // efface des cases, ou tout le fond
#include <defiler>       // fait glisser tout le fond

```

2. UNE VARIABLE — ECRANTITRE

```

uint8_t ecranTitre = 1; // 1 = le titre, 0 = l'écran d'après

```

3. UNE VARIABLE — D

```

uint8_t d = 0; // la caméra : 0 = elle regarde le début du décor

```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);          // le titre du 0.90, en orange
    texteGrand(4, 5, "JEU", 3);
    texte(2, 12, "APPUIE SUR START");

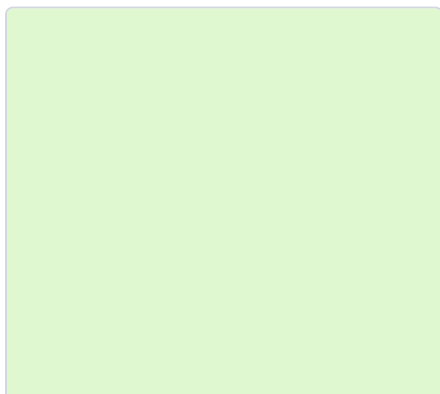
    while (true) {
        image();

        // Le changement d'écran du 0.90.1 : une seule fois, au premier appui sur START.
        if (ecranTitre == 1 && bouton(START)) {
            ecranTitre = 0;
            for (uint8_t y = 0; y < 18; y++) {
                effacer(0, y, 20);      // on vide les 18 lignes
            }
            texte(5, 8, "C EST PARTI");
        }

        // ---- NOUVEAU : tout l'écran glisse, seulement sur l'écran d'après
        //
        // d--          la caméra recule d'un pixel : 0, 255, 254, 253...
        // defiler(d, 0);
        //      | |
        //      | +- vers le bas : 0, on ne bouge pas en hauteur
        //      +----- vers la droite : d pixels
        //
        // la caméra recule → le décor semble avancer vers la DROITE
        if (ecranTitre == 0) {
            d--;
            defiler(d, 0);
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



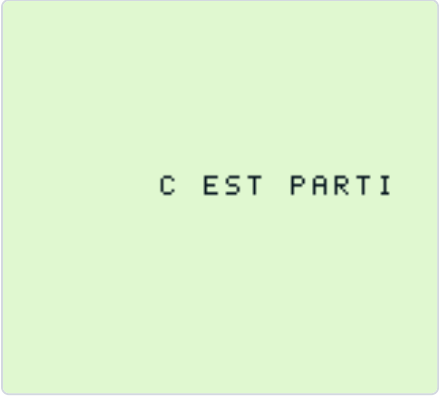
la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



C EST PARTI

START — l'écran a changé

5. Ce qu'on doit voir

Après START : C EST PARTI glisse vers la droite, sort de l'écran et revient par la gauche, sans fin.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ sur le titre, rien ne glisse
- ✓ après START, la caméra recule : d est passé sous 0 (255, 254...) (d = 228)
- ✓ et l'écran glisse vraiment (0 vers 228)

Ensuite : **0.90.5. L'écran titre — un bandeau qui passe** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++